Bitácora de Uso de Inteligencia Artificial (Copiloto de Apoyo Técnico)

En seis iteraciones, la IA aportó orientación técnica y el equipo detectó cada error o supuesto ajeno al caso y tomó la decisión final.

Iteración / Tema	Prompt o consulta realizada al asistente	Respuesta u orientación obtenida	Error, supuesto o ajuste detectado por el equipo	Decisión final del equipo
1. Elección de Patrón	*"¿Cómo puedo justificar técnicamente el uso de Domain Model frente a Transaction Script basándome en las reglas de inventario y devoluciones de una tienda de electrónica?"*	El asistente explicó los riesgos de la dispersión de lógica en scripts y sugirió estructurar la justificación alrededor de la protección de invariantes y reglas complejas.	La sugerencia inicial de la IA mezclaba conceptos de bases de datos relacionales en la capa de lógica.	El equipo filtró la explicación, descartó los aspectos de infraestructura y redactó la justificación enfocándose puramente en las reglas de negocio de TecnoStock.
2. Bounded Contexts y Agregados	*"Ayúdame a revisar si el modelo de inventario y seriales de TecnoStock debe separar el número de serie como un agregado independiente."*	Propuso modelar el `NumeroDeSerie` como un Aggregate Root para garantizar unicidad global y rastreabilidad de estados.	La propuesta inicial de la IA no contemplaba el almacenamiento de un historial de cambios para las auditorías.	Se ajustó el diseño para incorporar una entidad interna de eventos (`EventoSerial`), validando que cumpliera con las reglas del caso.
3. Service Layer	*"¿Cómo debo estructurar los métodos del Application Service para que coordinen casos de uso sin absorber reglas de negocio?"*	Entregó una lista de métodos orientados a intenciones de usuario (como `IniciarVenta`, `CrearApartado`, `SolicitarDevolucion`) separando la	La IA intentó colocar una validación de porcentajes de anticipo dentro del método del servicio.	Se rechazó esa ubicación y se reestructuró para que el servicio delegue esa validación estrictamente al interior del agregado de dominio

Iteración / Tema	Prompt o consulta realizada al asistente	Respuesta u orientación obtenida	Error, supuesto o ajuste detectado por el equipo	Decisión final del equipo
		orquestación.		correspondiente.
4. Repositorios y Unit of Work	*"¿Cuál es la mejor forma de definir interfaces de Repository para que el dominio no dependa de detalles de infraestructura SQL?"*	Sugirió crear contratos limpios basados en métodos del negocio (como `buscarPorFolio` u `obtenerStock`) y explicar el control transaccional por método.	La respuesta sugería usar un ORM específico directamente en los contratos de las entidades.	Se omitió dicha recomendación y el equipo diseñó las interfaces de repositorio desacopladas utilizando exclusivamente objetos y tipos del dominio.
5. Principios SOLID	*"¿Cómo puedo verificar si mi diseño cumple con el principio de Inversión de Dependencias (D) en la separación entre dominio e infraestructura?"*	Explicó que las capas superiores deben apuntar hacia abstracciones (interfaces) y no hacia implementaciones concretas de bases de datos.	El ejemplo genérico que dio la IA estaba orientado a frameworks web de Node.js/Python ajenos al proyecto escolar.	Se adaptó el concepto teórico para aplicarlo directamente a los contratos de repositorio y los servicios de aplicación de TecnoStock.
6. Sintaxis UML y PlantUML	*"Muéstrame un ejemplo rápido de sintaxis en PlantUML para representar clases con atributos privados, métodos y enumeraciones."*	Proporcionó un bloque de código básico de referencia con la estructura de clases y relaciones de agregación.	El código de ejemplo venía incompleto y no incluía los nombres específicos de las entidades de TecnoStock.	El equipo utilizó el ejemplo únicamente como guía de sintaxis y redactó manualmente todo el diagrama de clases y secuencias adaptado al caso real.